PRODUCT REQUIREMENTS DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

LoCor AI, Business Intelligence & Market Analysis Agent

Domain: AI for Economic Empowerment & Decision Intelligence

Field	Details
Team Name	Team 42 + 2
Team Members	Gilbert Gan Wei Xuan Colin Leong Hong Seng Gan Jayci Ow Ji Joe Tan Wen Zhe
Institution	Sunway University
Version	2.0
Date	26th of April 2026

Contents

1. Project Overview 4
Problem Statement 4
Target Domain 4
Proposed Solution Summary 4
2. Background & Business Objective 5
Background of the Problem 5
Importance of Solving This Issue 5
Strategic Fit & Impact 5
3. Product Purpose 6
3.1 Main Goal of the System 6
3.2 Intended Users (Target Audience) 6
4. System Functionalities 7
4.1 Description 7
4.2 Key Functionalities 7
Data Ingestion & Vectorisation 7
RAG-Powered AI Chat 7
Dashboard Generation 7
Market Insights 7
Competitive Pricing Strategy 8
Automated Report Generation & Distribution 8
4.3 AI Model & Prompt Design 8
4.3.1 Model Selection 8
4.3.2 Prompting Strategy 8
4.3.3 Context & Input Handling 9
4.3.4 Fallback & Failure Behaviour 9
5. User Stories & Use Cases 11
User Stories 11
Use Cases (Main Interactions) 11
UC-01: Data Upload & Vectorisation 11
UC-02: Dashboard & Insights Generation 11
UC-03: AI Chat 12
UC-04: Pricing Strategy 12
UC-05: Report Generation & Email 12
UC-06: Product Listing (for Pricing Page) 12
6. Features Included (Scope Definition) 14

7. Features Not Included (Scope Control) 15
8. Assumptions & Constraints 16
LLM Cost Constraint 16
Technical Constraints 16
Performance Constraints 16
User Input 17
9. Risks & Questions Throughout Development 18
10. References & Citations 19
11. AI Tools Used 21
12. AI Usage 21

1. Project Overview

Problem Statement

SME business owners, including retail operators, e-commerce sellers and service businesses, are drowning in data they cannot act on. Sales sheets, inventory logs and balance sheets exist in silos. Extracting meaning from them requires either expensive business analysts or significant time the owner does not have. Meanwhile, pricing decisions are made without competitive context and weekly reports are prepared manually with high labour overhead.

Target Domain

AI-powered business intelligence for small and medium enterprises (SMEs), with an initial focus on Malaysian retail and e-commerce operators selling on platforms such as Lazada.

Proposed Solution Summary

LoCor AI removes the friction of business analysis by allowing SME owners to upload their raw business documents, company description, inventory sheets, sales data and balance sheets and instantly receive a live AI dashboard, actionable market insights, a competitive pricing strategy and an automated weekly report delivered to their inbox. The owner's role shifts from spreadsheet analyst to strategic decision-maker.

2. Background & Business Objective

Background of the Problem

Most SME owners currently follow a fragmented, manual workflow: data is maintained across separate CSV or Excel files, insights are drawn from memory or gut feel and competitive pricing decisions are made with little real-time market intelligence. Engaging a consultant or a full-time data analyst is cost-prohibitive for businesses operating on lean margins. The result is slow decision cycles, missed restocking opportunities and reactive rather than proactive pricing.

Importance of Solving This Issue

Accelerating the business intelligence cycle from days (or weeks) to seconds enables SME owners to:

- Identify and act on inventory risks before stockouts or overstocking occur.
- Price competitively using live competitor data rather than assumption.
- Receive a professional weekly report without hiring an analyst.
- Ask natural-language questions about their own business data at any time.

Strategic Fit & Impact

LoCor AI is built on the Z.ai GLM-4.5-Flash [1] model and ChromaDB [2] for vector storage, with a FastAPI [4] backend and React [12] frontend. It is positioned as a lightweight, deployable intelligence layer that sits on top of data an SME owner already has, requiring no change to their existing workflows, only an upload.

3. Product Purpose

3.1 Main Goal of the System

To provide SME owners with an AI-powered business partner that converts raw operational data into real-time insights, competitive intelligence and automated reporting, without requiring technical skills or specialist knowledge.

3.2 Intended Users (Target Audience)

User Type	Primary Need
SME Business Owner / Founder	Understand business health at a glance; receive weekly strategy recommendations
Product / Operations Manager	Track inventory status, flag reorder needs, monitor category performance
E-commerce Seller (Lazada)	Make data-driven pricing decisions using live competitor data
Finance / Accounts Team	Access automated Word and Excel reports without manual compilation

4. System Functionalities

4.1 Description

LoCor AI operates as a Retrieval-Augmented Generation (RAG) engine. On upload, all business documents are parsed, converted to text representations, chunked and stored as vector embeddings in ChromaDB [2]. Every downstream AI operation, chat, insights, dashboard, reports, pricing, retrieves semantically relevant chunks from this store before sending them to the language model as context. This ensures all AI outputs are grounded in the user's actual business data.

4.2 Key Functionalities

Data Ingestion & Vectorisation

Accepts PDF, DOCX or TXT company descriptions. Accepts CSV or XLSX spreadsheets for Inventory, Sales and Balance Sheets. All data is parsed via PyMuPDF [8] and pandas [13], then stored across four named ChromaDB collections [2]: Company_Description, Inventory_Sheets, Sales_Sheets and Balance_Sheets.

RAG-Powered AI Chat

A persistent, multi-turn chat assistant grounded in the user's business data. Chat history is stored in SQLite [5] and prepended to every request. Responses are streamed token-by-token. The assistant is prompted to behave as a professional SME business consultant with 15+ years of experience.

Dashboard Generation

Generates a structured JSON dashboard including: four key KPI stats (Monthly Revenue, Total Sales, Top Category Revenue, Items Needing Reorder), sales breakdown by category with percentage shares, top 5 products by units sold and per-category trend analysis with actionable pricing suggestions.

Market Insights

Generates a structured JSON insights report including: per-category trend direction (up/down/neutral), per-insight action labels (Buy More / Reduce / Hold / Watch) and a full inventory health matrix flagging each product as critical (below 20% capacity), excess (above 90%) or ok.

Competitive Pricing Strategy

Accepts a product name from the user. Retrieves the product's own data from ChromaDB (cost, margin, stock, sales volume). Fetches live competitor listings from Lazada via API. Encodes competitor data using TOON format for token efficiency. Streams a structured pricing analysis including: market position, supply gap identification, exact RM price recommendation, new margin calculation and Lazada listing optimisation advice. Results are cached in Redis [3] [3] per product to avoid redundant API calls.

Automated Report Generation & Distribution

On demand, generates a structured weekly business proposal in Markdown (converted to .docx) and a full inventory change spreadsheet (.xlsx) with row-level change tracking (added / removed / modified / unchanged) and cell-level conditional highlighting. Both files are emailed as attachments to the configured recipient via the Resend API [11].

4.3 AI Model & Prompt Design

4.3.1 Model Selection

LoCor AI uses Z.ai's GLM-4.5-Flash [1] as its core language model. GLM-4.5-Flash [1] was selected for the following reasons:

- Speed: Flash-class models return responses significantly faster than full-scale models, which is critical for streaming endpoints where users expect near-real-time output.
- Cost: Lower cost per token enables the product to remain viable at SME-scale usage without aggressive rate limiting.
- JSON reliability: GLM-4.5-Flash reliably produces structured JSON output when prompted appropriately, which is essential for the dashboard, insights and Excel generation pipelines.
- Context window: Sufficient to accommodate RAG-retrieved chunks plus prompt templates across all five use cases.
- Thinking disabled: The product disables chain-of-thought (thinking: disabled) across all calls to minimise token overhead and latency.

4.3.2 Prompting Strategy

LoCor AI uses a multi-strategy approach depending on the use case:

- Zero-shot structured output: For dashboard and insights generation, the model is given a detailed system prompt specifying the exact JSON schema required, with field-level rules. No examples are provided; the model's instruction-following is relied upon entirely. This is appropriate because the output format is rigid and consistent and few-shot examples would consume excessive context tokens.
- Role-conditioned generation: For chat and document generation, the model is given a rich persona (SME business consultant, 15 years experience) and instructed on tone, scope and constraints. This grounds the outputs without consuming tokens on examples.
- Data-conditioned generation: For pricing strategy, the prompt is dynamically assembled from structured user product data and retrieved competitor data, both encoded in TOON (json-toon) format for token efficiency. The model is given explicit numbered sub-tasks to answer in sequence, ensuring comprehensive coverage of the analysis.

4.3.3 Context & Input Handling

All user-uploaded documents are chunked at the point of vectorisation. ChromaDB retrieves the top 3 most semantically similar chunks per query per collection, filtered by a cosine distance threshold of 1.8. Chunks that exceed this threshold are discarded. For the pricing strategy endpoint, competitor product data is encoded using the json-toon serialisation format before being injected into the prompt, reducing token count by approximately 25-30% compared to raw JSON. There is no hard token limit enforced at the input level; instead, chunk retrieval naturally limits context size. Inputs that are semantically irrelevant to the query are excluded by the distance filter.

4.3.4 Fallback & Failure Behaviour

The system implements the following failure handling:

- JSON parse failure: Dashboard and insights endpoints use regex-based JSON cleaning (stripping markdown fences) before parsing. If parsing fails after cleaning, the error is propagated to the caller with a 500 status.
- Empty retrieval: If ChromaDB returns no results above the distance threshold, the AI is called with no injected context. The model will respond based on its general knowledge, which may be less specific but will not cause a system failure.
- Pricing API failure: If all configured pricing APIs (currently Lazada) fail to respond, the streaming endpoint returns a graceful plain-text error message: 'All pricing APIs failed to

respond.' A mock data flag (USE_MOCK_PRICING) can be set in server.py for development or fallback use.

- Model off-topic response: There is currently no automated hallucination detector. The system relies on structured prompting to constrain outputs. For JSON-producing endpoints, a failed parse serves as an implicit quality gate.

5. User Stories & Use Cases

User Stories

ID	As a...	I want to...	So that...
US-01	SME Owner	Upload my sales, inventory and balance sheets in a single step	I can have all my business data available to the AI immediately
US-02	SME Owner	View a live dashboard of my key business KPIs	I can understand my business health without opening a spreadsheet
US-03	SME Owner	Ask the AI questions about my inventory and sales in plain English	I can get specific answers without writing formulas or queries
US-04	E-commerce Seller	Enter a product name and receive a competitor pricing analysis	I can price competitively based on live market data
US-05	Operations Manager	Receive a weekly Word report and Excel inventory update by email	I can share professional reports with stakeholders without manual preparation
US-06	SME Owner	View AI-generated market insights with buy/hold/reduce recommendations	I know which products to invest in and which to clear

Use Cases (Main Interactions)

UC-01: Data Upload & Vectorisation

The user navigates to the Upload page and submits four files: a company description (PDF/DOCX/TXT) and three spreadsheets (Inventory, Sales, Balance Sheet in CSV/XLSX). The DocumentParser extracts text from each file. The VectorStore chunks and stores each document in its corresponding ChromaDB collection. The system returns a success status.

UC-02: Dashboard & Insights Generation

The user navigates to the Dashboard or Insights page. The system queries ChromaDB for 'sales trends inventory performance' across all four collections. Retrieved chunks are assembled into a

context block and sent to GLM-4.5-Flash [1] with the relevant structured-output prompt. The model returns a JSON object which is parsed and rendered in the UI.

UC-03: AI Chat

The user types a question on the Chat page. The system retrieves semantically relevant chunks from ChromaDB. The full SQLite chat history plus the new message and retrieved context are sent to GLM-4.5-Flash [1]. The response is streamed token-by-token to the UI and saved to SQLite on completion.

UC-04: Pricing Strategy

The user enters a product name on the Pricing page. The system checks Redis for a cached result. If absent, it retrieves the product's own data from Inventory_Sheets and Sales_Sheets collections. Live competitor data is fetched from Lazada. Both datasets are encoded in TOON format, assembled into a structured prompt and sent to GLM-4.5-Flash [1]. The analysis streams to the UI and is cached in Redis on completion.

UC-05: Report Generation & Email

The user triggers report generation. The system retrieves full business context from ChromaDB. GLM-4.5-Flash [1] generates a Markdown business proposal and a JSON inventory change array. The proposal is converted to .docx using python-docx [9]. The inventory array is written to .xlsx using openpyxl [10] with conditional row and cell highlighting. Both files are emailed as attachments via Resend API [11].

UC-06: Product Listing (for Pricing Page)

The Pricing page loads a product list by calling the /products endpoint. The server queries Inventory_Sheets and Sales_Sheets from ChromaDB, parses the pandas DataFrame repr format and returns a structured list of products with margin, stock status and sales data.

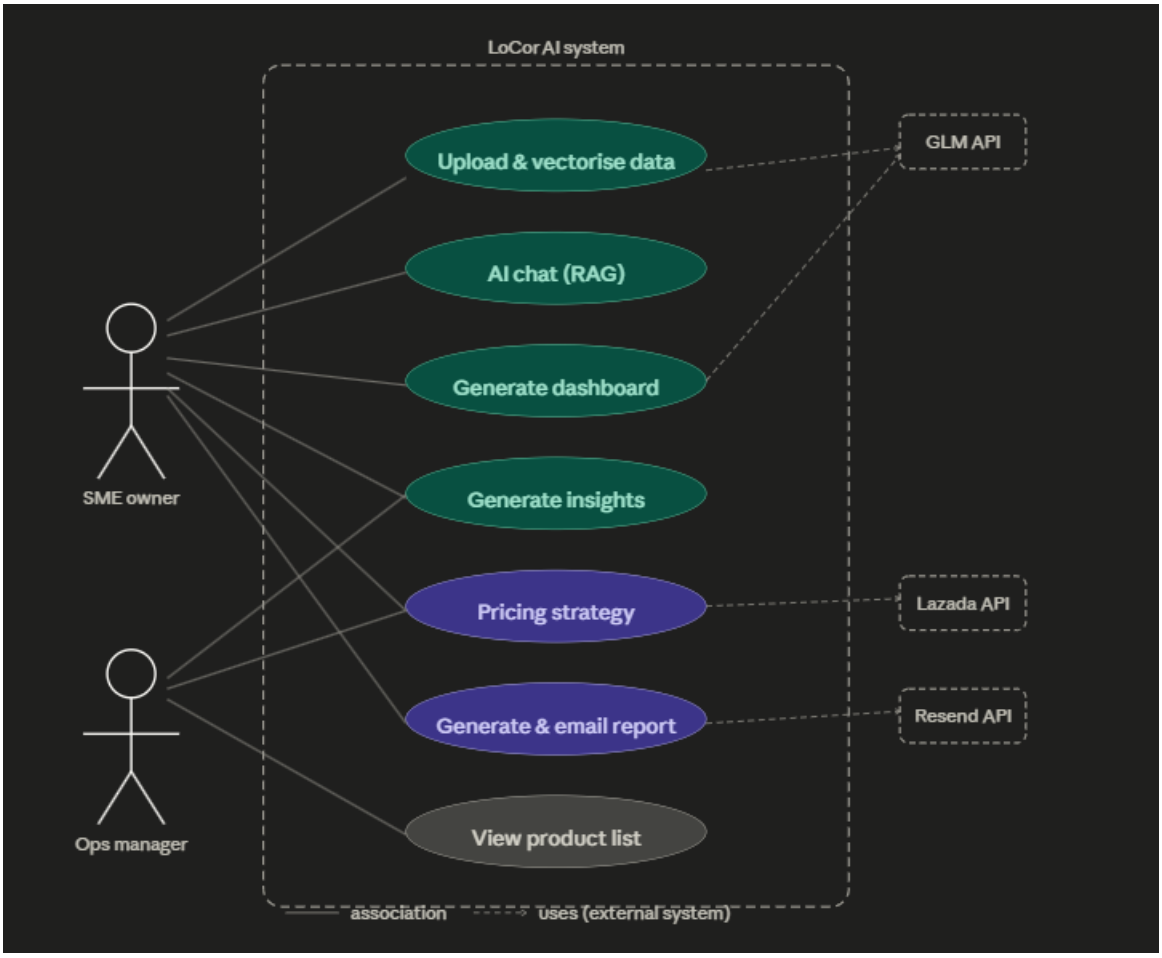

Fig. 1. User use case

6. Features Included (Scope Definition)

- Multi-format document upload: PDF, DOCX, TXT for descriptions; CSV, XLSX for spreadsheets.
- Vector-based semantic search over user business data via ChromaDB with distance-threshold filtering.
- Persistent, multi-turn AI chat grounded in business data with SQLite-backed history and SSE streaming.
- AI-generated dashboard with KPI stats, category breakdowns, top products and trend analysis.
- AI-generated market insights with per-category trend direction and per-product action recommendations.
- Live competitive pricing analysis using Lazada API data, with Redis caching and TOON token optimisation.
- Automated .docx report generation (weekly business proposal in structured Markdown).
- Automated .xlsx inventory update with row-level change tracking and conditional cell highlighting.
- Email delivery of generated reports and spreadsheets via Resend API.
- Product listing endpoint with computed margin, stock status and matched sales volume.
- Mock pricing data mode for development without live API credentials.

7. Features Not Included (Scope Control)

- Multi-user / team access: The system is designed for a single business owner. No user authentication, role management or per-user data isolation is implemented.
- Real-time data sync: The system operates on uploaded snapshots. It does not connect directly to live POS systems, ERP platforms or e-commerce dashboards.
- Multi-platform competitor data: Only Lazada is currently integrated. Shopee, Amazon or other platforms are out of scope.
- Automated scheduling: Reports are generated on demand. There is no cron job or scheduled weekly delivery.
- Frontend mobile optimisation: The React frontend is not optimised for mobile screen sizes in the current version.
- Export to React / Swift / other frameworks: Code export is not a feature; the product generates business documents, not UI components.
- Production-grade authentication: No OAuth, JWT or session management is implemented.

8. Assumptions & Constraints

LLM Cost Constraint

Average token usage per user session is estimated as follows:

- Chat interaction (per turn): 1,500 input tokens (history + RAG context + prompt) + 500 output tokens = 2,000 tokens.
- Dashboard / Insights generation: 2,000 input tokens + 1,500 output tokens = 3,500 tokens.
- Pricing strategy: 2,500 input tokens (TOON-encoded competitor data + user data + prompt) + 1,500 output tokens = 4,000 tokens.
- Report generation: 2,500 input + 2,000 output (document) + 2,000 input + 2,000 output (Excel) = 8,500 tokens.

Estimated total per heavy session: 18,000 tokens. Key cost-management decisions: (1) TOON encoding reduces competitor data token count by 25-30%; (2) Redis caching of pricing results means repeat queries for the same product incur zero model cost; (3) thinking mode is disabled on all calls; (4) GLM-4.5-Flash is used instead of a full-scale model, offering significantly lower per-token pricing.

Technical Constraints

- Code export: Not supported. The product generates .docx and .xlsx business documents only.
- Vector DB persistence: ChromaDB stores data to disk at ./chroma_db. Data persists between server restarts but is local to the deployment machine. No cloud vector DB is used.
- Redis dependency: The pricing strategy cache requires a running Redis instance [3] on localhost:6379. If Redis is unavailable, the cache layer will fail silently and every pricing request will hit the live API.
- File size: No hard file size limit is enforced, but very large spreadsheets may exceed GLM's context window when their full DataFrame repr is used as a RAG chunk.

Performance Constraints

- Streaming latency: Chat and pricing responses begin streaming within 1-3 seconds of the request, depending on model load.

- Report generation: Document and Excel generation is sequential (document first, then Excel), meaning total report generation time can reach 15-30 seconds for data-rich inputs.
- ChromaDB query: Each dashboard or insights generation queries all four collections, resulting in up to 12 ChromaDB queries per request.

User Input

- The system requires human-initiated document uploads before any AI feature can function. Without uploaded data, RAG context will be empty and AI responses will be generic.
- The system is not production-hardened. Human review of generated reports before distribution to external stakeholders is recommended.

9. Risks & Questions Throughout Development

Risk	Description	Status / Mitigation
Data Privacy	User business data (sales, margins, inventory) is sensitive. It is stored locally in ChromaDB without encryption.	Accepted for MVP. Production deployment should add at-rest encryption and access controls.
ChromaDB Staleness	Each upload appends new documents to existing collections. Old, superseded data remains and may be retrieved alongside newer data, degrading answer quality.	Identified risk. A document versioning or collection-reset mechanism should be added before production.
Pandas Repr Parsing	The /products and pricing endpoints parse ChromaDB-stored pandas DataFrame repr strings using regex. This is fragile and may break with unusual column names or multi-word values.	Known fragility. Should be replaced with structured JSON storage at ingest time.
Lazada API Reliability	The pricing strategy feature depends on a third-party API. Rate limits, API changes or downtime will break the feature.	Mitigated by: mock data mode flag, graceful error stream message and Redis cache reducing call frequency.
Model Hallucination in Reports	GLM may generate plausible-sounding but inaccurate business recommendations, especially when RAG context is thin.	Prompts instruct the model not to invent data. Human review before report distribution is recommended.
TOON Format Compatibility	json-toon is a non-standard serialisation format. Future model updates may handle it less reliably.	Monitor output quality. Fall back to JSON if issues arise.
Redis Unavailability	If Redis is down, every pricing request hits the Lazada API, risking rate limits and increased cost.	Document Redis as a hard dependency. Add health check on startup.

10. References & Citations

All external tools, libraries, APIs and resources referenced in this document are cited below in IEEE format, in compliance with UMHackathon 2026 submission rules.

#	Citation (IEEE Format)
[1]	Z.AI, "GLM-4.5-Flash," Z.AI. [Online]. Available: [Z.AI API documentation URL]. [Accessed: Apr. 2026].
[2]	Chroma, "ChromaDB," Chroma. [Online]. Available: https://www.trychroma.com. [Accessed: Apr. 2026].
[3]	Redis Ltd., "Redis," Redis. [Online]. Available: https://redis.io. [Accessed: Apr. 2026].
[4]	S. Ramirez, "FastAPI," FastAPI. [Online]. Available: https://fastapi.tiangolo.com. [Accessed: Apr. 2026].
[5]	SQLite Consortium, "SQLite," SQLite. [Online]. Available: https://www.sqlite.org. [Accessed: Apr. 2026].
[8]	Artifex Software, "PyMuPDF," PyMuPDF. [Online]. Available: https://pymupdf.readthedocs.io. [Accessed: Apr. 2026].
[9]	python-docx Contributors, "python-docx," python-docx. [Online]. Available: https://python-docx.readthedocs.io. [Accessed: Apr. 2026].
[10]	openpyxl Contributors, "openpyxl," openpyxl. [Online]. Available: https://openpyxl.readthedocs.io. [Accessed: Apr. 2026].
[11]	Resend Inc., "Resend," Resend. [Online]. Available: https://resend.com. [Accessed: Apr. 2026].
[12]	Meta Open Source, "React," React. [Online]. Available: https://react.dev. [Accessed: Apr. 2026].
[13]	The pandas Development Team, "pandas," pandas. [Online]. Available: https://pandas.pydata.org. [Accessed: Apr. 2026].

11. AI Tools Used

In compliance with UMHackathon 2026 rules requiring proper citation of all AI tools used, the following tools were referenced during the preparation of this document:

#	Citation (IEEE Format)
[A1]	Z.AI, "GLM-4.5-Flash," Z.AI. [Online]. Available: https://docs.z.ai/guides/llm/glm-4.5 [Accessed: Apr. 2026].
[A2]	Anthropic, "Claude AI Assistant," Anthropic, 2025. [Online]. Available: https://claude.ai. [Accessed: Apr. 2026].

12. AI Usage

In compliance with UMHackathon 2026 submission requirements, all AI tools used during the development of LoCor AI are declared below.

Z.AI GLM-4.5-Flash is the core reasoning engine built into LoCor AI itself. It handles RAG-powered chat, business insight generation, competitor pricing analysis and automated report generation. This is a product component, not a development aid.

Claude (Anthropic) was referred to for writing ideas and ideation suggestions when preparing the PRD, SAD and QATD. All written content was drafted by our team based on our own understanding of the system. AI was not used to write the final content directly. Diagrams in this document were also generated with the assistance of Claude (Anthropic).

During development, we also referred to AI for guidance in a few specific areas:

1. Frontend: UI component structure and layout ideas for the React 19 pages
2. Prompt engineering: prompt structure and system persona design for the GLM integration
3. JSON mapping: mapping backend API responses to frontend components
4. Code cleanup: code readability and refactoring suggestions

In all of these cases, the ideas, decisions and actual implementation were done by our team. AI was used as a reference, not as a replacement for our own work.